Technical Review

Smart i-LAB AIR1 All-Zones

Room-level occupancy estimation from two camera views, per-zone labels, shared environmental sensing, and readiness gates

pctiope/smart-i-lab-testbed

May 2026

Technical Review Scope

What is reviewed

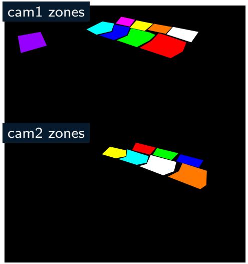
- One shared model for AIR-1 zones 1–15.
- Two-camera CV labels, long-form rows, feature contract, training, serving, and readiness.
- Dashboard usefulness before and after a production model pointer exists.

What is explicitly out of scope

- Zone 5 power and mmWave model inputs.
- Zone 5 BSG rollout and CI/CD claims, except as contrast.
- Populated environment values, camera URLs, credentials, live logs, and generated run metrics.

Zones 1–15 | Two cameras | Long-form rows | No power/mmWave

Problem Setting



Tracked camera-zone masks define AIR1 coverage.

Research setting

AIR1 asks for room-level occupancy over zones 1–15. The evidence is not one desk, one camera, or one sensor stream: two camera views, per-zone masks, shared AIR-1 readings, SEN55 readings, and readiness gates all shape the result.

- Camera coverage differs by zone and timestamp.
- Missing camera coverage should stay unknown, not become vacancy.
- The dashboard must remain useful while production inference is gated.

Research Questions

1. Can one shared temporal model represent zones 1–15 from long-form `timestamp + zone_id` rows?
2. Can per-zone CV labels stay null when coverage is missing instead of becoming false negatives?
3. Can `zone_id` support grouping, audits, splits, and display without entering the model input?
4. Can the all-zones dashboard expose camera freshness and readiness before a production model is promoted?

The deck expands the applied-research story into a technical review of source acquisition, labels, features, windowing, model architecture, validation, serving, deployment, and limitations.

System Scope And Ownership

Package scope

- Package: AIR1 all-zones package-local CSV/Parquet path.
- Runtime state roots: `data/`, `model/`, `logs/`.
- Training target: `occupied`.
- Row key: `timestamp + zone_id`.
- No root Zone 5 BSG DuckDB path is inherited.

Service ownership

- Camera trackers own per-zone count evidence.
- Aggregator owns 10-second zone buckets.
- Live collector owns joined CSV/Parquet rows.
- Trainer and promoter own candidate and production pointers.
- FastAPI owns readiness, history, streams, and per-zone dashboard output.

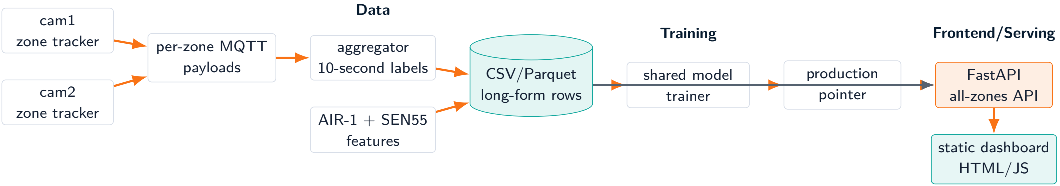
Contract boundary

AIR1 is not Zone 5 plus more zones. It remains package-local CSV/Parquet, has no BSG/power/mmWave inheritance, and keeps `zone_id` out of model input.

Package Structure

Path	Review role
air1_all_zones/feature_contract.py	Declares valid zones, raw inputs, missing indicators, time columns, target, sample interval, and lookback choices.
rtsp_zone_tracker.py	Camera-zone tracker that emits per-zone count payloads.
air1_all_zones/occupancy_mqtt_aggregator.py	Merges per-zone messages into 10-second timestamp + zone_id CV rows.
air1_all_zones/build_cv_training_data.py	Joins features and labels while excluding Table 16 and preserving null labels.
air1_all_zones/training.py	Shared 1D CNN, Optuna search, timestamp-grouped splits, and artifact metadata.
web_app/main.py	FastAPI all-zones dashboard endpoints and camera freshness health.
tests/test_all_zones_contract.py	Regression tests for feature exclusion, per-zone labels, payload parsing, and readiness behavior.

End-To-End Runtime Flow



Data: track + aggregate + join

Training: shared model + pointer

Frontend: FastAPI static UI

Architectural boundary

AIR1 remains package-local CSV/Parquet and does not inherit Zone 5 BSG, Vite, power, or mmWave assumptions.

Active Services

Unit	Owned behavior
`air1-all-zones-person-counter-cam1.service`	Runs cam1 zone tracker and writes cam1 per-zone evidence.
`air1-all-zones-person-counter-cam2.service`	Runs cam2 zone tracker and writes cam2 per-zone evidence.
`air1-all-zones-mqtt-aggregator.service`	Converts per-zone messages into `timestamp + zone_id` 10-second labels.
`air1-all-zones-sen55-collector.service`	Buckets SEN55 readings and writes CSV/Parquet air-quality rows.
`air1-all-zones-live-collector.service`	Appends joined all-zones CSV/Parquet rows and metadata.
`air1-all-zones-live-app.service`	Serves health, history, SSE, per-camera MJPEG, and all-zones current state.
`air1-all-zones-trainer.timer/service`	Runs scheduled oneshot retraining and promotion when readiness allows.

Two-Camera Coverage

cam1 coverage

- Zones: 4, 9, 10, 11, 12, 13, 14, 15.
- Table 16 can appear as excluded camera context.
- Emits per-zone counts for covered zones.

cam2 coverage

- Zones: 1, 2, 3, 5, 6, 7, 8.
- Complements cam1 coverage.
- Missing coverage is not converted into a zero count.

Review boundary

The model contract accepts zones 1–15. Table 16 is intentionally excluded until the feature contract changes.

Coverage And Row-Shape Chart

Zones 1–15 | Table 16 excluded | timestamp + zone id | nullable target

Chart scope

The chart shows mask ownership and row contract shape only; it does not encode live detections, occupancy counts, or generated training metrics.

Per-Zone MQTT Payloads

Payload meaning

- Counts are keyed by zone for a camera observation.
- Payloads may include unlabeled zones.
- Camera IDs and label source are kept for auditability.
- Table 16 is ignored by the model-label path.

Aggregator behavior

- Buckets rows at the 10-second sample cadence.
- Writes unique `timestamp + zone_id` keys.
- Preserves nullable counts for unlabeled coverage.
- Outputs CSV and Parquet label tables.

Per-zone | Camera-audited | Nullable | Table 16 excluded

Label Contract

Target semantics

- Target: `occupied`.
- Key: `timestamp + zone_id`.
- Positive rule: median `occupancy_count >= 1`.
- Negative rule: median count is below 1.
- Missing label rule: `occupied = null`.

What null protects

- Camera not covering a zone at a timestamp.
- Missing per-zone payload evidence.
- Late or absent MQTT messages.
- Validation from treating unknown as confidently empty.

Contract test coverage

Regression tests assert that missing per-zone labels stay null and that Table 16 is excluded from cleaned labels.

Long-Form Training Rows

Column family	Role
`timestamp`	10-second sample boundary.
`zone_id`	Row key, grouping handle, split/audit/display field; not a model feature.
AIR-1 raw	`temp`, `rh`, `co2`, `pm25`.
SEN55 raw	PM, temperature, humidity, VOC, and NOx columns.
Missing indicators	`*_missing` channels for AIR-1 and SEN55 raw inputs; included in AIR1 model inputs.
Time features	Hour and day-of-week sin/cos cycles.
CV target	`occupied`; nullable when CV evidence is absent.

A single timestamp normally expands into up to 15 rows, one per valid model zone.

AIR1 Feature Contract

Model inputs

- AIR-1 environmental columns.
- Shared SEN55 columns.
- Missing indicators for those raw columns.
- Deterministic time features.

Excluded from model input

- `zone_id`: grouping, audit, split, display only.
- Power fields: Zone 5-only source family.
- mmWave fields: Zone 5-only source family.
- Table 16 labels and rows.

AIR-1 SEN55 Missing indicators Time

AIR1 Versus Zone 5 Contract

Topic	AIR1 All-Zones	Zone 5
Data plumbing	Package-local CSV/Parquet.	Root BSG DuckDB/Silver training input.
Row shape	Long-form `timestamp` + `zone_id`.	One row per Zone 5 timestamp.
Valid scope	Zones 1–15; Table 16 excluded.	Zone 5 only.
Target	`occupied`.	`zone_occupied`.
Power input	Excluded.	`power_s5`.
mmWave input	Excluded.	`mmwave_s5` plus recency.
Missingness	`*_missing` included as model inputs.	Missingness retained for diagnostics, not model inputs.

Why the contrast matters

Porting Zone 5 runtime safety patterns does not mean importing Zone 5 BSG plumbing or sensor assumptions into AIR1.

Package-Local Collector And Joined Snapshots

Live collector inputs

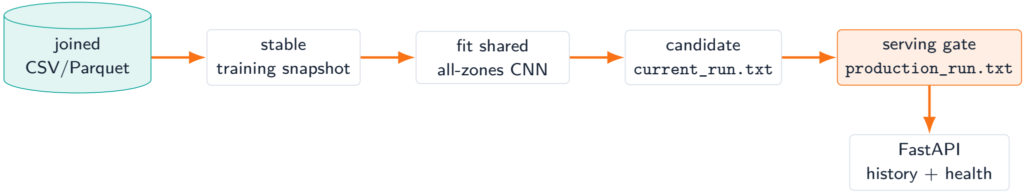
- AIR-1 histories for the valid zone device set.
- Package-local SEN55 bucket table.
- Package-local all-zones CV label table.
- Recent backfill to catch late labels.

Live collector outputs

- Joined training CSV.
- Joined training Parquet.
- Joined-table metadata JSON.
- Rolling rows with generated artifacts kept out of git.

The joined CSV/Parquet table is the shared evidence source; the AIR1 trainer snapshots it before fitting and does not read Zone 5 BSG tables.

Production Pointer Readiness



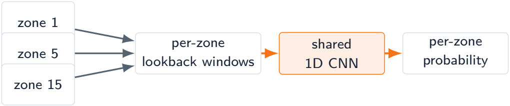
Readiness semantics

Missing `production_run.txt` means production inference is not ready, but camera freshness, labels, and dashboard health can still be useful.

Shared Rolling Windows

Window construction

- Candidate lookbacks: 15, 60, and 180 minutes.
- Rows are grouped by zone for sequence construction.
- Timestamp grouping keeps all zones for one timestamp in the same split.
- `zone_id` is carried with windows for audit/display, not as a channel.



Model shape

The model learns shared temporal patterns across zones while split policy prevents timestamp leakage across partitions.

1D CNN Architecture

Stage	Source behavior
Input	AIR-1, SEN55, missing indicators, and time channels over a selected lookback.
Conv blocks	2 or 3 `Conv1d` blocks; base channels grow by block and cap at 256.
Kernel choices	First and late kernels are searched independently over compact odd sizes.
Normalization	Optional batch normalization after convolution.
Activation	Search over ReLU, GELU, and SiLU.
Pooling	Max-pooling after block 2 or after each late block.
Global pooling	Adaptive average or max pooling to one temporal summary.
Classifier	Dense layer, activation, dropout, final single-logit output.

TunableZoneOccupancyCNN   Shared weights   Binary output

Optuna And Training Loop

Search space

- Lookback, batch size, learning rate, weight decay.
- Conv block count, channels, kernel sizes.
- Activation, normalization, pooling, global pooling.
- Dense units, dropout, scheduler, patience.

Training mechanics

- Optimizer: AdamW.
- Class imbalance handled with positive weight from train labels.
- Schedulers: none, cosine, or reduce-on-plateau.
- Objective: mean validation PR-AUC across eligible folds.
- Final model trains on all pre-test rows.

Shared model implication

AIR1 searches one model configuration for all zones rather than fitting isolated per-zone models.

Split And Coverage Policy

Strict path

- Latest calendar day is held out as blind test.
- Pre-test dates form one-day rolling validation folds.
- Timestamp groups keep all zones for one timestamp together.
- Valid lookbacks must fit train and validation windows.

Coverage evidence

- Per-zone label coverage is checked.
- Both-class windows are required for meaningful validation.
- Rejected lookbacks and under-covered dates are recorded.
- Bootstrap fallback is first-production recovery only.

Timestamp grouping | Rolling calendar | Per-zone coverage | Both-class windows

Validation And Promotion Gates

Model evidence

- Blind-test PR-AUC and mean CV PR-AUC thresholds.
- Brier score and log-loss ceilings.
- Positive windows, positive buckets, and positive events.
- Progressive strict-fold maturity as production matures.

Software evidence

- Smoke loading rejects incompatible artifacts.
- Same-window non-regression applies when production is comparable.
- Promotion writes the production pointer atomically.
- Promotion failure does not stop collection.

Readiness And Promotion Funnel

CSV/Parquet row snapshot → coverage + null label checks → timestamp-grouped split policy → quality gates threshold policy → atomic production pointer → FastAPI static dashboard

Package-local rows | Coverage-aware labels | Shared model | Pointer-gated serving

Operational reading

AIR1 readiness is allowed to improve without importing Zone 5 BSG/Vite assumptions; promotion remains a local artifact and pointer decision.

Contract Tests And Smoke Checks

High-value tests

- Model features exclude `zone_id`, power, and mmWave.
- One timestamp emits all 15 valid zone rows.
- Per-zone labels join on `timestamp + zone_id`.
- Table 16 is excluded and unlabeled zones stay null.

Runtime checks

- Smoke loading validates model and feature compatibility.
- Web-app checks cover health and route behavior.
- Camera freshness remains observable without active inference.
- Production pointer state is reported rather than hidden.

Review implication

The all-zones contract is enforced both in dataset code and in tests, not just in documentation.

FastAPI Endpoint Surface

Endpoint	Purpose
`/`	Static all-zones dashboard shell.
`/api/health`	Readiness, data-source, pointer state, history size, camera config, camera status, and latest error.
`/api/current`	Latest all-zones event, or 503 when no history exists.
`/api/history?n=`	Bounded recent event history.
`/api/stream`	Server-sent events for live dashboard updates.
`/api/video/{camera_id}.mjpg`	Camera-specific MJPEG stream for cam1 or cam2.
`/api/video.mjpg`	Compatibility MJPEG route using the primary camera stream.

Interface constraint

The presentation change does not alter public API, schema, services, filenames, or Make targets.

Health Payload Semantics

Health can prove

- The app process is responsive.
- Camera grabbers are configured and report freshness.
- History size and current readiness are visible.
- The latest error explains blockers such as pointer absence.

Health does not prove

- A production model has already been promoted.
- Every zone has current labels.
- Sensor/API features are complete.
- A candidate model passed promotion.

The dashboard is allowed to be operationally useful before production inference is ready, as long as readiness is explicit.

Dashboard Usefulness While Inference Is Gated

Useful before promotion

- cam1 and cam2 MJPEG feeds show view freshness.
- Per-zone CV truth can be displayed independently from model probability.
- Health payload reports missing production pointer or source blockers.

Applied-research implication

AIR1 separates observation quality from model readiness. A dashboard can support data collection and diagnosis before the first promotable all-zones model exists, as long as it does not present missing inference as validated occupancy.

Collect Audit Gate Promote

Deployment And Operations

Persistent runtime

- User-systemd services own camera loops, aggregator, SEN55 collector, live collector, app, and trainer timer.
- Generated data, logs, and model runs remain package-local and untracked.
- AIR1 does not use root `run_bsg_live_collector.sh` or `run_bsg_trainer.sh`.
- The trainer lock prevents overlapping full fits.

Operational readiness

- Enable trainer timer only after collection and label coverage are mature enough.
- Check pointers, status JSON, service state, and health routes before declaring readiness.
- Keep Zone 5 BSG, power, and mmWave detail out of AIR1 claims.

Manual-first rule

AIR1 automation should follow a successful proof run; scheduled training should not obscure whether coverage, split, or model gates are the real blocker.

Limitations And Threats To Validity

Data and label threats

- Camera occlusion and mask coverage vary by zone and view.
- Missing camera coverage creates null labels and reduces usable evidence.
- Table 16 exclusion limits model scope to zones 1–15.
- Calendar-window maturity can block strict rolling validation.

Operational threats

- Production pointer absence means inference is not ready even when collection is alive.
- Sensor/API availability affects environmental feature freshness.
- Training quality depends on enough occupied and unoccupied windows per split.
- Shared modeling may hide zone-specific failure modes without per-zone audits.

Roadmap

Near-term hardening

- Mature per-zone label coverage before treating all-zones inference as production-ready.
- Promote only after strict-window evidence, smoke checks, and contract tests pass.
- Keep camera freshness, null label rates, and production pointer state visible.

Research extensions

- Add per-zone reliability summaries for reviewers and operators.
- Compare shared-model errors across camera coverage groups.
- Extend contract only after a source-grounded reason to include new fields.

Source Anchors

Topic	Primary source paths
Pipeline map	`PIPELINE.md`
Feature and target contract	`air1_all_zones/feature_contract.py`
CV labels and masks	`rtsp_zone_tracker.py`, `masks/`
Joined training table	`air1_all_zones/build_cv_training_data.py`
Model and training	`air1_all_zones/training.py`
Promotion	`air1_all_zones/promote_model.py`
Runtime readiness	`web_app/main.py`
Contract tests	`tests/test_all_zones_contract.py`

Closing point: AIR1 is an all-zones, two-camera occupancy study whose null labels and readiness gates are part of the method.